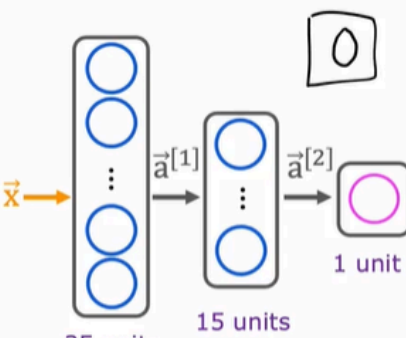


Neural Network Training

TensorFlow implementation

Train a Neural Network in TensorFlow



```
import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(units=25, activation='sigmoid'),
    Dense(units=15, activation='sigmoid'),
    Dense(units=1, activation='sigmoid'),
])

from tensorflow.keras.losses import BinaryCrossentropy
model.compile(loss=BinaryCrossentropy())

model.fit(X, Y, epochs=100)
```

Given set of (x, y) examples
How to build and train this in code?

①
②
③
epochs: number of steps in gradient descent

- Step 1: is to specify the model, which tells TensorFlow how to compute for the inference.
- Step 2: compiles the model using a specific loss function.
- Step 3: is to train the model.

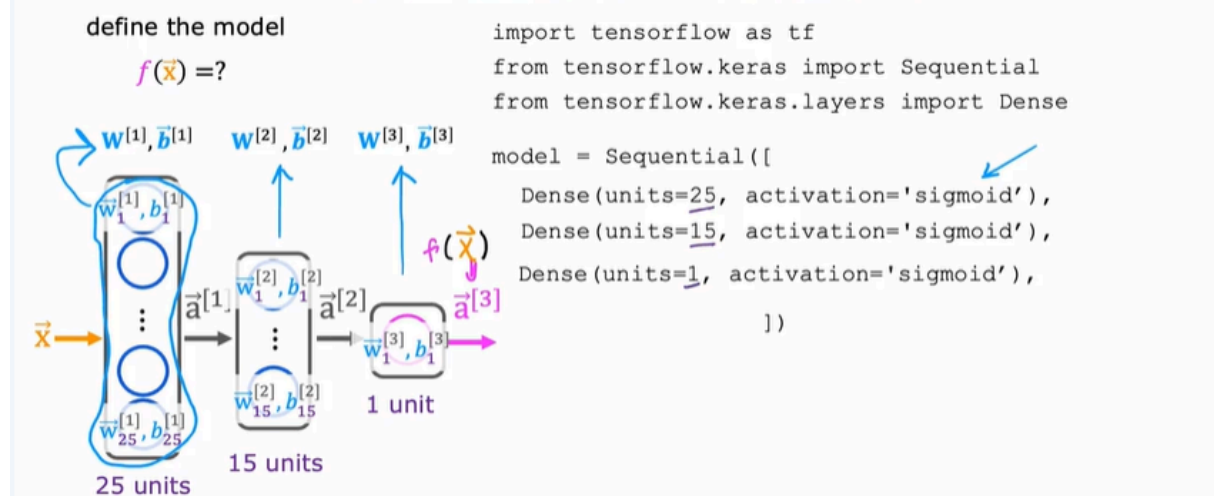
Training Details

Model Training Steps TensorFlow

① specify how to compute output given input x and parameters w, b (define model) $f_{\bar{w}, b}(\bar{x}) = ?$	logistic regression <pre>z = np.dot(w, x) + b f_x = 1 / (1 + np.exp(-z))</pre>	neural network <pre>model = Sequential([Dense(...), Dense(...), Dense(...)])</pre>
② specify loss and cost $L(f_{\bar{w}, b}(\bar{x}), y)$ 1 example $J(\bar{w}, b) = \frac{1}{m} \sum_{i=1}^m L(f_{\bar{w}, b}(\bar{x}^{(i)}), y^{(i)})$	logistic loss <pre>loss = -y * np.log(f_x) -(1-y) * np.log(1-f_x)</pre>	binary cross entropy <pre>model.compile(loss=BinaryCrossentropy())</pre>
③ Train on data to minimize $J(\bar{w}, b)$	<pre>w = w - alpha * dj_dw b = b - alpha * dj_db</pre>	<pre>model.fit(X, y, epochs=100)</pre>

Step 1: Create the model

1. Create the model



The first step, specify how to compute the output given the input x and parameters w and b . This code snippet specifies the entire architecture of the neural network. It tells you that there are 25 hidden units in the first hidden layer, then the 15 in the next one, and then one output unit and that we're using the sigmoid activation value.

Based on this code snippet, we know also what are the parameters $w^{[1]}, b^{[1]}$ though the first layer parameters of the second layer and parameters of the third layer. This code snippet specifies the entire architecture of the neural network and therefore tells TensorFlow everything it needs in order to

compute the output $\vec{a}^{[3]}$ or $f(\vec{x})$ as a function of the input $\vec{x}$ and the parameters $w^{[l]}$ and $b^{[l]}$.

Step 2: Loss and cost functions

2. Loss and cost functions

handwritten digit classification problem → binary classification

$$L(f(\vec{x}), y) = -y \log(f(\vec{x})) - (1 - y) \log(1 - f(\vec{x}))$$

Compare prediction vs. target

logistic loss
also known as binary cross entropy

$$J(\mathbf{W}, \mathbf{B}) = \frac{1}{m} \sum_{i=1}^m L(f(\vec{x}^{(i)}), y^{(i)})$$

$\mathbf{W}^{[1]}, \mathbf{W}^{[2]}, \mathbf{W}^{[3]}$ $\mathbf{B}^{[1]}, \mathbf{B}^{[2]}, \mathbf{B}^{[3]}$ $f_{\mathbf{W}, \mathbf{B}}(\vec{x})$

```

model.compile(loss= BinaryCrossentropy())
regression
(predicting numbers and not categories)
model.compile(loss= MeanSquaredError())
    
```

from tensorflow.keras.losses import
BinaryCrossentropy

from tensorflow.keras.losses import
MeanSquaredError

K Keras

In the second step, you have to specify what is the loss function. That will also define the cost function we use to train the neural network. For the handwritten digit classification problem where images are either of a zero or a one the most common by far, loss function to use is this one is actually the same loss function as what we had for logistic regression is

$$L(f(\vec{x}), y) = -y \log(f(\vec{x})) - (1 - y) \log(1 - f(\vec{x}))$$

, where y is the ground truth label, sometimes also called the target label y , and $f(\vec{x})$ is now the output of the neural network.

In TensorFlow, this is called the binary cross-entropy loss function. Where does that name come from? Well, it turns out in statistics this function on top is called the cross-entropy loss function, so that's what cross-entropy means, and the word binary just reemphasizes or points out that this is a binary classification problem because each image is either a zero or a one.

The syntax is to ask TensorFlow to compile the neural network using this loss function. Another historical note, carers was originally a library that had developed independently of TensorFlow is actually totally separate project from

TensorFlow. But eventually it got merged into TensorFlow, which is why we have `tf.keras library.losses` dot the name of this loss function.

- By the way, I don't always remember the names of all the loss functions and TensorFlow, but I just do a quick web search myself to find the right name and then I plug that into my code.

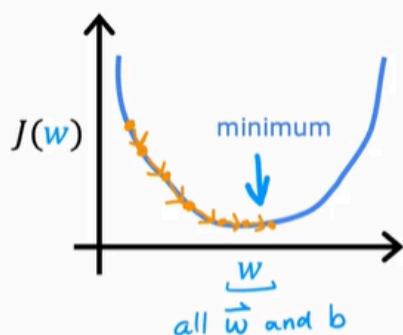
Having specified the loss with respect to a single training example, TensorFlow knows that it costs you want to minimize is then the average, taking the average over all m training examples of the loss on all of the training examples. Optimizing this cost function will result in fitting the neural network to your binary classification data. In case you want to solve a regression problem rather than a classification problem. You can also tell TensorFlow to compile your model using a different loss function.

For example, if you have a regression problem and if you want to minimize the squared error loss. Here is the squared error loss (`MeanSquaredError()`). The loss with respect to if your learning algorithm outputs $f(\vec{x})$ with a target or ground truth label of y , that's $1/2$ of the squared error. Then you can use this loss function in TensorFlow, which is to use the maybe more intuitively named mean squared error loss function. Then TensorFlow will try to minimize the mean squared error.

In this expression, I'm using $J(W, B)$ to denote the cost function. The cost function is a function of all the parameters into neural network. You can think of W as including $W^{[1]}$, $W^{[2]}$, $W^{[3]}$. All the W parameters and the entire new network and be as including $\vec{b}^{[1]}$, $\vec{b}^{[2]}$, and $\vec{b}^{[3]}$. If you are optimizing the cost function respect to w and b , if we tried to optimize it with respect to all of the parameters in the neural network. Up on top as well, I had written $f(\vec{x})$ as the output of the neural network, but we can also write $f_{W,B}(\vec{x})$ if we want to emphasize that the output of the neural network as a function of x depends on all the parameters in all the layers of the neural network. That's the loss function and the cost function.

Step 3: Gradient Descent

3. Gradient descent



```
repeat {  
     $w_j^{[l]} = w_j^{[l]} - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$   
     $b_j^{[l]} = b_j^{[l]} - \alpha \frac{\partial}{\partial b_j} J(\vec{w}, b)$   
} Compute derivatives  
for gradient descent  
using "backpropagation"  
  
model.fit(X, y, epochs=100)
```

Finally, you will ask TensorFlow to minimize the cross-function. You might remember the gradient descent algorithm from the first course.

If you're using gradient descent to train the parameters of a neural network, then you are repeatedly, for every layer l and for every unit j , update $w_j^{[l]}$ according to $w_j^{[l]}$ minus the learning rate alpha times the partial derivative with respect to that parameter of the cost function J of w and b and similarly for the parameters b as well. After doing, say, 100 iterations of gradient descent, hopefully, you get to a good value of the parameters.

In order to use gradient descent, the key thing you need to compute is these partial derivative terms. What TensorFlow does, and, in fact, what is standard in neural network training, is to use an algorithm called backpropagation in order to compute these partial derivative terms. TensorFlow can do all of these things for you. It implements **backpropagation** all within this function called `fit`. All you have to do is call `model.fit`, `x`, `y` as your training set, and tell it to do so for 100 iterations or 100 epochs. In fact, what you see later is that TensorFlow can use an algorithm that is even **a little bit faster than gradient descent**, and you'll see more about that later this week as well. Now, I know that we're relying heavily on the TensorFlow library in order to implement a neural network. One pattern I've seen across multiple ideas is as the technology evolves, libraries become more mature, and most engineers will use libraries rather than implement code from scratch.

Neural network libraries

Use code libraries instead of coding "from scratch"



Good to understand the implementation
(for tuning and debugging).